Name: Abdulaziz Mohammed Alkhlawi
ID: 2240006095

Deep Learning for NLP

Deep Learning is a type of Machine Learning that uses **Neural Networks with multiple layers** to learn patterns from data.

Deep Learning can be used in NLP tasks such as:

- Text Classification
- Sentiment Analysis
- Question Answering

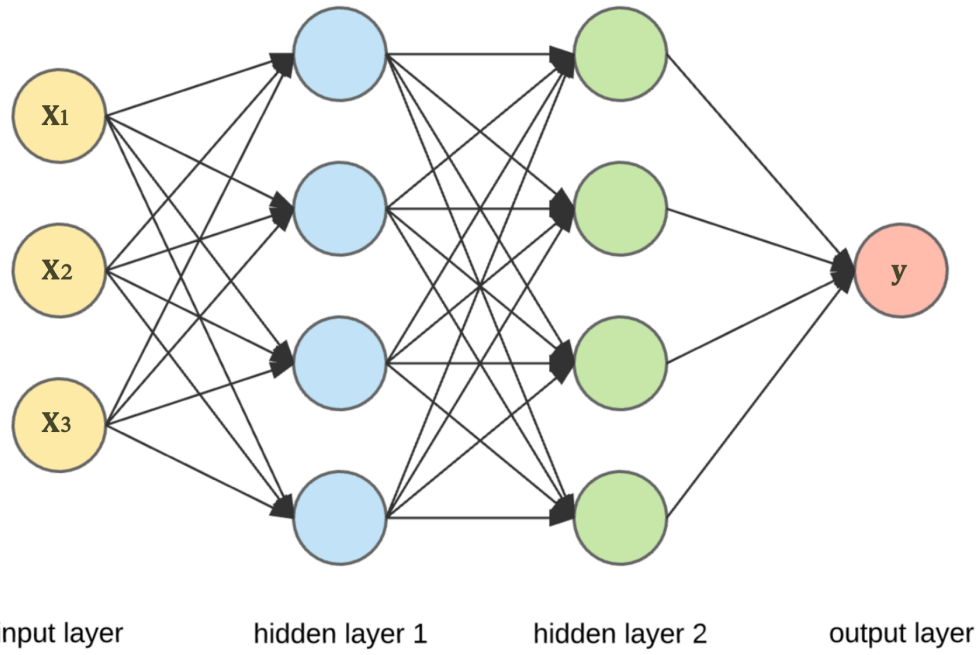
In this lab, we will build a **deep learning model** to classify Yelp restaurant reviews as:

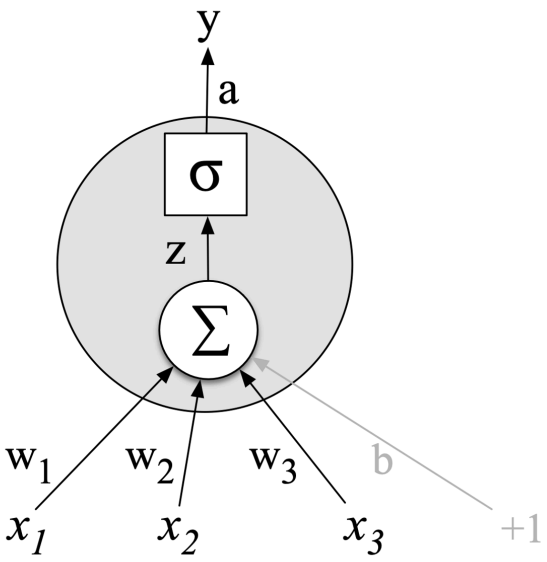
- **0 = Negative**
- **1 = Positive**

Completed lab solution. Original instructions: NLP_Lab6.

Run the cells from top to bottom. Results below were generated by executing this notebook. The `dataset` folder contains the original data needed by this lab. Missing Python packages are installed by the setup cell; pretrained model downloads need internet on their first run. Random seeds are fixed where supported.

```
# Install only missing dependencies into the active Jupyter Python environment.
import importlib.util, subprocess, sys, os, random
from pathlib import Path
packages = {'numpy': 'numpy', 'pandas': 'pandas', 'matplotlib': 'matplotlib', 'sklearn': 'scikit-learn',
'nltk': 'nltk', 'torch': 'torch', 'gensim': 'gensim'}
missing = [package for module, package in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing])
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['USE_TF'] = '0'  # Transformers examples use PyTorch.
os.environ['USE_FLAX'] = '0'
import numpy as np
import pandas as pd
from IPython.display import display, Markdown, HTML
random.seed(42)
np.random.seed(42)
pd.set_option('display.max_colwidth', 90)
pd.set_option('display.max_columns', 10)
DATA = Path('dataset')
assert DATA.exists() or 6 in (7, 8, 9), 'Open this notebook from its Lab folder.'
```





1. Load the Yelp Dataset

Each row contains:

- a restaurant review
- its sentiment label

```
df = pd.read_csv(DATA / '07-yelp-dataset.txt', sep='\t', names=['review','label'])
print('Number of reviews:',len(df))
display(df.head())
```

```
Number of reviews: 1000
```

	review	label
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday off Rick Steve recommendation and loved it.	1
4	The selection on the menu was great and so were the prices.	1

```
print(df["label"].value_counts())
```

```
label
1    500
0    500
Name: count, dtype: int64
```

2. Preprocessing the Dataset

```
import re

def preprocess_text(text):

    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

df["review"] = df["review"].apply(preprocess_text)
df.head()
```

	review	label
0	wow loved this place	1
1	crust is not good	0
2	not tasty and the texture was just nasty	0
3	stopped by during the late may bank holiday off rick steve recommendation and loved it	1
4	the selection on the menu was great and so were the prices	1

3. Split the Data

We divide the dataset into:

- **80% training data** — used to train the model
- **20% testing data** — used to evaluate the model

`stratify=y` keeps the proportion of positive and negative reviews similar in both sets.

```python
from sklearn.model_selection import train_test_split

X = df["review"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

print("Training reviews:", len(X_train))
print("Testing reviews:", len(X_test))
```

```
Training reviews: 800
Testing reviews: 200
```

4. Convert Text to TF-IDF Vectors

A neural network cannot directly understand text.

TF-IDF converts each review into a vector of numbers.

```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=1000, lowercase=True)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

```
Training TF-IDF shape: (800, 1000)
Testing TF-IDF shape: (200, 1000)
```

5. Convert the Data to PyTorch Tensors

PyTorch neural networks work with **tensors**.

```python
import torch
import torch.nn as nn

torch.manual_seed(42)
torch.set_num_threads(2)

# Convert TF-IDF data to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train_tfidf.toarray())
X_test_tensor = torch.FloatTensor(X_test_tfidf.toarray())

y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)
```

```
C:\Users\Azoz\AppData\Local\Temp\ipykernel_34748\3224988659.py:11: UserWarning: The given NumPy array is not writable, and PyTorch does not support non-writable tensors. This means writing to this tensor will result in undefined behavior. You may want to copy the array to protect its data or make it writable before converting it to a tensor. This type of warning will be suppressed for the rest of this program. (Triggered internally at C:\actions-runner\_work\pytorch\pytorch\torch\csrc\utils\tensor_numpy.cpp:214.)
  y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)
```

6. Build a Simple Neural Network

Our network has:

1. **Input layer** — receives the TF-IDF features
2. **Hidden layer** — learns useful patterns

3. **ReLU activation** — adds non-linearity
4. **Output layer** — produces one value for binary classification

We do **not** put `Sigmoid` inside the model because `BCEWithLogitsLoss` applies it internally in a numerically stable way.

```
input_size = X_train_tensor.shape[1]
# torch.size([800, 1000])

model = nn.Sequential(
    nn.Linear(input_size, 64),   # Input layer
    nn.ReLU(),

    nn.Linear(64, 32),           # Hidden layer
    nn.ReLU(),

    nn.Linear(32, 1)             # Output layer
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=1000, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=1, bias=True)
)
```

7. Choose the Loss Function and Optimizer

- **BCEWithLogitsLoss** is used for binary classification. To measure the model's mistakes.
- **Adam** is responsible for changing the model's weights so that the loss becomes smaller.

```
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

8. Train the Neural Network

One **epoch** means the model has seen the full training dataset once.

For every epoch:

1. Make predictions
2. Calculate the loss
3. Calculate gradients with backpropagation
4. Update the weights

```
epochs = 10

for epoch in range(epochs):
    model.train()

    optimizer.zero_grad()  # Clear gradients from the previous epoch.
    # 1. Forward pass
    outputs = model(X_train_tensor)

    # 2. Calculate loss
    loss = loss_function(outputs, y_train_tensor)

    # 3. Backpropagation
```

```
    loss.backward()

    # 4. Update model weights
    optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch + 1}/{epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 5/10 - Loss: 0.6242
Epoch 10/10 - Loss: 0.3577
```

9. Evaluate the Model

The model outputs raw scores called **logits**.

We apply `sigmoid` to convert them to probabilities between 0 and 1.

- Probability **≥ 0.5** → **Positive (1)**
- Probability **< 0.5** → **Negative (0)**

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model.eval()

with torch.no_grad():
    test = model(X_test_tensor)
    test_probabilities = torch.sigmoid(test)
    predictions = (test_probabilities >= 0.5).int().numpy().flatten()

accuracy = accuracy_score(y_test, predictions)

print(f"Test Accuracy: {accuracy:.3f}")
```

```
Test Accuracy: 0.755
```

10. Classification Report and Confusion Matrix

The classification report shows:

- **Precision**
- **Recall**
- **F1-score**

The confusion matrix shows how many reviews were classified correctly and incorrectly.

```
print("Classification Report:")
print(classification_report(y_test, predictions, target_names=["Negative", "Positive"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

```
Classification Report:
              precision    recall  f1-score   support

    Negative       0.81      0.67      0.73       100
    Positive       0.72      0.84      0.77       100

    accuracy                           0.76       200
   macro avg       0.76      0.76      0.75       200
weighted avg       0.76      0.76      0.75       200

Confusion Matrix:
```

```
[[67 33]
 [16 84]]
```

Task 1 – Represent Reviews Using Word2Vec

In the previous part of the lab, TF-IDF was used to convert each review into numerical features. Now, use Word2Vec instead.

Requirements: Tokenize the training and testing reviews. Train a Word2Vec model using the training reviews. Use: vector_size = 200 window = 5 min_count = 1 sg = 1

Convert each review into a single vector. Convert the resulting vectors into PyTorch tensors.

```
from gensim.models import Word2Vec
train_tokens = [text.split() for text in X_train]
test_tokens = [text.split() for text in X_test]
embedding = Word2Vec(train_tokens,vector_size=200,window=5,min_count=1,sg=1,
                     seed=42,workers=1,epochs=20)
def review_vector(tokens):
    vectors = [embedding.wv[word] for word in tokens if word in embedding.wv]
    return np.mean(vectors,axis=0) if vectors else np.zeros(200,dtype=np.float32)
train_w2v = np.stack([review_vector(tokens) for tokens in train_tokens])
test_w2v = np.stack([review_vector(tokens) for tokens in test_tokens])
# Standardize using training statistics only to improve neural optimization.
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(train_w2v)
train_w2v_tensor = torch.tensor(scaler.transform(train_w2v),dtype=torch.float32)
test_w2v_tensor = torch.tensor(scaler.transform(test_w2v),dtype=torch.float32)
print('Word2Vec train/test tensors:',train_w2v_tensor.shape,test_w2v_tensor.shape)
print('Training vocabulary size:',len(embedding.wv))
```

```
Word2Vec train/test tensors: torch.Size([800, 200]) torch.Size([200, 200])
Training vocabulary size: 1787
```

Task 2 – Build a Different Neural Network

Use the Word2Vec vectors from Task 1 as the input to a new neural network. Your network must contain: An input layer Three hidden layers ReLU activation functions One output neuron Use the following architecture:

Input

↓

128 neurons

↓

64 neurons

↓

32 neurons

↓

1 output neuron

Train the model using: , BCEWithLogitsLoss , Adam optimizer , Learning rate = 0.001 , 15 epochs. Then calculate the test accuracy.

```
torch.manual_seed(42)
w2v_network = nn.Sequential(nn.Linear(200,128),nn.ReLU(),
    nn.Linear(128,64),nn.ReLU(),nn.Linear(64,32),nn.ReLU(),nn.Linear(32,1))
print(w2v_network)
w2v_loss = nn.BCEWithLogitsLoss()
w2v_optimizer = torch.optim.Adam(w2v_network.parameters(),lr=0.001)
from torch.utils.data import TensorDataset, DataLoader
loader = DataLoader(TensorDataset(train_w2v_tensor,y_train_tensor),batch_size=32,
    shuffle=True,generator=torch.Generator().manual_seed(42))
history=[]
for epoch in range(15):
    w2v_network.train()
    total_loss=0.0
    for features,labels in loader:
        w2v_optimizer.zero_grad()
        loss=w2v_loss(w2v_network(features),labels)
        loss.backward()
        w2v_optimizer.step()
        total_loss+=loss.item()*len(features)
    history.append(total_loss/len(X_train))
    print(f'Epoch {epoch+1:2d}/15 - loss: {history[-1]:.4f}')
w2v_network.eval()
with torch.no_grad():
    w2v_predictions=(torch.sigmoid(w2v_network(test_w2v_tensor))>=0.5).int().numpy().ravel()
print('Word2Vec test accuracy:',round(accuracy_score(y_test,w2v_predictions),3))
print(classification_report(y_test,w2v_predictions,target_names=['Negative','Positive'],zero_division=0))
print('Confusion matrix:',confusion_matrix(y_test,w2v_predictions),sep='\n')
import matplotlib.pyplot as plt
plt.plot(range(1,16),history,marker='o')
plt.xlabel('Epoch');plt.ylabel('Training BCE loss');plt.title('Word2Vec neural network')
plt.tight_layout();plt.show()
```

```
Sequential(
  (0): Linear(in_features=200, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=64, bias=True)
  (3): ReLU()
  (4): Linear(in_features=64, out_features=32, bias=True)
  (5): ReLU()
  (6): Linear(in_features=32, out_features=1, bias=True)
)
Epoch  1/15 - loss: 0.6757
Epoch  2/15 - loss: 0.6602
Epoch  3/15 - loss: 0.6500
Epoch  4/15 - loss: 0.6336
Epoch  5/15 - loss: 0.6296


Epoch  6/15 - loss: 0.6115


Epoch  7/15 - loss: 0.5922
Epoch  8/15 - loss: 0.5655
Epoch  9/15 - loss: 0.5424
Epoch 10/15 - loss: 0.5238
Epoch 11/15 - loss: 0.5036

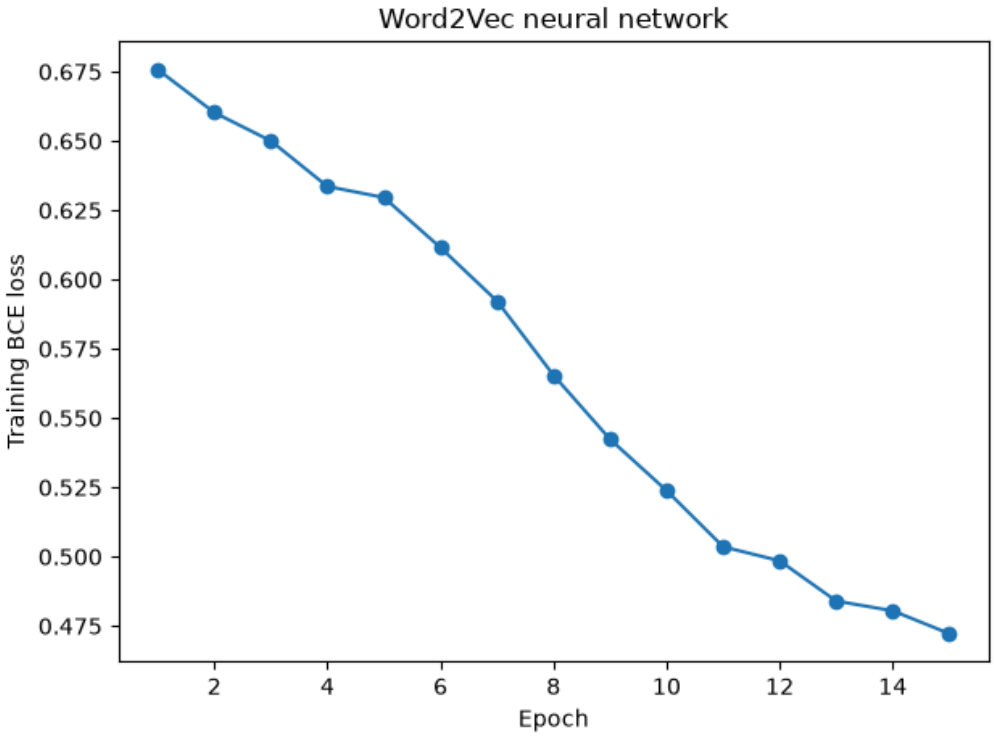

Epoch 12/15 - loss: 0.4985
Epoch 13/15 - loss: 0.4840
Epoch 14/15 - loss: 0.4805
Epoch 15/15 - loss: 0.4722
Word2Vec test accuracy: 0.73
              precision    recall  f1-score   support

    Negative       0.74      0.71      0.72       100
    Positive       0.72      0.75      0.74       100

    accuracy                           0.73       200
   macro avg       0.73      0.73      0.73       200
weighted avg       0.73      0.73      0.73       200

Confusion matrix:
```

```
[[71 29]
 [25 75]]
```



Design notes

Word2Vec is trained exclusively on training reviews. Mean pooling produces one 200-dimensional vector per review, and a zero vector handles a review with no known tokens. Standardization is also fitted only on training vectors. The new network has the required three hidden layers (128, 64, 32), one output logit, Adam at 0.001 and exactly 15 epochs. Both networks reset gradients at every optimization step. A small corpus limits the quality of learned word embeddings.